Semut Adaptive Architecture: A Unified Adaptive Memory Architecture for Intelligent Systems

Timothy Lawrence Sitorus
Semut LLC

July 2, 2026

Abstract

Modern intelligent systems increasingly rely on multiple components, including large language models (LLMs), retrieval systems, workflow engines, knowledge stores, and user feedback mechanisms. While each component has advanced significantly, they are often developed as independent subsystems with limited coordination. Workflow engines optimize execution, retrieval systems manage information access, and feedback mechanisms are frequently confined to isolated evaluation pipelines. As a result, continual adaptation across the entire system remains fragmented. This paper introduces the *Semut Adaptive Architecture*, a unified systems architecture that models intelligent systems through three interconnected memory structures: *Artifact Memory*, *Action Graph*, and *Feedback Memory*. Artifact Memory preserves persistent contextual information such as documents, source code, conversations, and structured knowledge. Action Graph represents executable actions and their dependencies as an evolving graph of operational behavior. Feedback Memory continuously records human evaluations, execution outcomes, environmental observations, and system performance, enabling future decisions to incorporate accumulated experience. Rather than replacing existing reasoning systems, the proposed architecture is reasoning-engine independent. Rule-based systems, large language models, future learning algorithms, and human operators can all utilize the same adaptive memory framework. Retrieved artifacts, historical actions, and prior feedback collectively provide contextual evidence for selecting subsequent actions. The outcomes of execution are subsequently incorporated back into all three memory structures, forming a continuous adaptive loop. The primary contribution of this work is not a new reasoning algorithm, but a unified architectural abstraction that integrates artifacts, actions, and feedback into a coherent adaptive memory system. This abstraction provides a common interface for continual learning, workflow execution, retrieval, and human-in-the-loop collaboration across heterogeneous intelligent systems. Potential applications include personal AI assistants, software engineering agents, enterprise automation, robotics, and multi-agent coordination.

1 Introduction

Recent advances in artificial intelligence have significantly expanded the capabilities of software systems. Large Language Models (LLMs) now enable natural language interaction with documents, software applications, databases, and external services. At the same time, retrieval-augmented generation (RAG), workflow orchestration, persistent memory systems, and agent frameworks have become common architectural components in modern AI applications. Rather than relying on a single model, intelligent systems increasingly consist of multiple interacting subsystems responsible for reasoning, retrieval, execution, and evaluation.

Although these components have individually advanced, they are often designed as largely independent subsystems. Retrieval systems focus on identifying relevant information. Workflow engines manage executable actions and their dependencies. Persistent storage maintains documents and structured knowledge. Human feedback mechanisms evaluate system outputs, while reasoning engines determine subsequent actions. Because these components frequently evolve independently, experience accumulated in one subsystem is not consistently incorporated into the others. As a result, continual adaptation across the entire system remains fragmented.

This fragmentation becomes increasingly important as AI assistants perform longer and more complex tasks. Intelligent systems must not only retrieve information, but also remember previous actions, evaluate outcomes, adapt future behavior, and coordinate interactions across heterogeneous software environments. Existing architectures often require multiple disconnected representations of system state, making it difficult to maintain a coherent history of both human and machine activities.

This paper proposes the *Semut Adaptive Architecture*, a unified systems architecture that organizes adaptive intelligence around three complementary memory structures: *Artifact Memory*, *Action Graph*, and *Feedback Memory*. Artifact Memory preserves persistent contextual information such as documents, conversations, source code, and structured data. Action Graph models executable actions together with their dependencies and observed outcomes. Feedback Memory records evaluations from humans, software systems, and environmental observations that influence future decision making. Rather than treating these memories as isolated data stores, the proposed architecture continuously updates all three through a shared adaptive feedback loop.

An important characteristic of the proposed architecture is its independence from any particular reasoning engine. Rule-based systems, Large Language Models, future learning algorithms, and even human operators can all utilize the same adaptive memory framework. The architecture therefore separates persistent knowledge representation from reasoning itself, allowing reasoning engines to evolve independently while sharing accumulated experience through a common memory interface.

The objective of this work is not to introduce a new reasoning algorithm or memory representation. Instead, it presents an architectural abstraction that

unifies artifacts, executable actions, and accumulated feedback into a coherent adaptive system. By treating these components as interconnected memories rather than isolated subsystems, intelligent systems may better support continual learning, workflow execution, human-in-the-loop collaboration, and long-term adaptation.

The primary contributions of this paper are:

- A unified architectural framework integrating Artifact Memory, Action Graph, and Feedback Memory.
- A reasoning-engine-independent design applicable to rule-based systems, LLMs, and future intelligent agents.
- A continuous adaptive feedback loop that updates all memory structures following execution.
- An architectural perspective for constructing intelligent assistants capable of long-term adaptation across heterogeneous software environments.

The remainder of this paper is organized as follows. Section **??** reviews existing approaches to memory architectures, workflow systems, and AI agents. Section **??** discusses the limitations of fragmented adaptive systems. Section 4 introduces the Semut Adaptive Architecture and its three memory structures. Section **??** describes the adaptive feedback loop connecting reasoning, execution, and memory updates. Finally, the paper discusses implementation considerations, potential applications, limitations, and future research directions.

2 Related Work

Modern intelligent systems have evolved through several complementary research directions, including retrieval systems, workflow orchestration, agent frameworks, persistent memory architectures, and large language models. While these approaches address important aspects of intelligent behavior, they often emphasize different representations of knowledge, execution, and adaptation.

Retrieval-Augmented Generation (RAG) extends language models with external knowledge retrieval, allowing responses to incorporate information beyond the model's training data. Vector databases and semantic search systems provide efficient mechanisms for retrieving documents and structured information. These systems primarily focus on artifact retrieval rather than representing executable actions or accumulating operational feedback.

Workflow orchestration platforms such as Zapier, n8n, Apache Airflow, and similar systems model business processes as directed execution graphs. Their primary objective is reliable task execution, dependency management, and automation across heterogeneous software platforms. Although execution histories and logs are maintained, they are generally treated as operational records rather than adaptive memories that influence future reasoning.

Recent AI agent frameworks, including LangGraph, AutoGen, CrewAI, and related systems, combine reasoning with external tools and workflow execution. These frameworks demonstrate increasingly sophisticated planning capabilities by integrating language models with structured execution. However, persistent memory, workflow representation, and feedback mechanisms are frequently implemented as separate components whose interactions depend on application-specific implementations.

Persistent memory architectures for intelligent assistants have similarly received increasing attention. Conversation histories, vector memories, episodic memories, and knowledge graphs provide mechanisms for preserving contextual information across interactions. Nevertheless, these memories often emphasize information retrieval while leaving action histories and feedback propagation as independent concerns.

Consequently, modern intelligent systems commonly maintain multiple representations of state. Documents and conversations reside in knowledge stores, executable procedures are represented within workflow engines, and evaluations are collected through separate logging or human feedback pipelines. Although each subsystem addresses a specific aspect of intelligent behavior, relatively few architectural frameworks explicitly treat artifacts, actions, and feedback as equally important adaptive memories within a unified representation.

Table 1 summarizes this architectural perspective.

Table 1: High-level comparison of representative intelligent system architectures. The comparison illustrates broad architectural emphasis rather than exhaustive implementation details.

System	Artifacts	Actions	Feedback	Unified Adaptive Memory
RAG Systems	✓	–	Partial	–
Workflow Engines	Partial	✓	Partial	–
Knowledge Graphs	✓	Partial	–	–
AI Agent Frameworks	✓	✓	Partial	Partial
Semut Adaptive Architecture	✓	✓	✓	✓

Unlike existing categories, the Semut Adaptive Architecture does not replace retrieval systems, workflow engines, or reasoning models. Instead, it proposes a common architectural abstraction in which Artifact Memory, Action Graph, and Feedback Memory are treated as interconnected adaptive components. This separation between adaptive memory and reasoning enables multiple reasoning engines, including rule-based systems, large language models, and future learning algorithms, to operate over the same evolving memory structure.

3 Problem Statement

Modern intelligent systems increasingly combine multiple components, including reasoning engines, retrieval systems, workflow orchestration, persistent storage, and human feedback mechanisms. Although each component performs an

important function, they frequently maintain independent representations of system state. As intelligent assistants become capable of performing longer and more complex tasks, maintaining separate representations introduces several architectural challenges.

First, contextual knowledge is often separated from executable behavior. Documents, conversations, and structured information are typically stored within retrieval systems or databases, while executable procedures are represented independently inside workflow engines or application logic. Consequently, reasoning engines must repeatedly reconstruct relationships between retrieved information and available actions.

Second, execution histories are commonly treated as operational logs rather than adaptive memories. Workflow engines record completed tasks and execution status for monitoring and debugging purposes, but these histories are not consistently incorporated into future reasoning. As a result, successful execution patterns and previous failures are difficult to reuse across different tasks.

Third, feedback frequently exists outside the primary decision-making architecture. Human evaluations, software-generated metrics, environmental observations, and execution outcomes are often collected by separate monitoring or evaluation systems. Although these signals provide valuable information about system performance, they rarely participate directly in future retrieval, planning, or execution.

These architectural separations become increasingly significant for long-running intelligent assistants. Systems operating across heterogeneous software environments must continuously coordinate documents, executable actions, user preferences, environmental observations, and accumulated experience. Without a unified adaptive representation, maintaining consistent long-term behavior becomes progressively more difficult as interactions increase.

This work therefore considers the following architectural question:

> How can artifacts, executable actions, and accumulated feedback be represented within a unified adaptive memory architecture while remaining independent of the underlying reasoning engine?

Rather than proposing a new reasoning algorithm, this paper investigates an architectural abstraction that organizes these three complementary forms of memory into a continuous adaptive feedback loop. The following section introduces the proposed Semut Adaptive Architecture and describes the responsibilities of Artifact Memory, Action Graph, and Feedback Memory.

4 Semut Adaptive Architecture

The Semut Adaptive Architecture models intelligent systems as a continuous interaction between reasoning, execution, and adaptive memory. Rather than treating retrieval systems, workflow engines, and feedback mechanisms as independent subsystems, the proposed architecture organizes them around three

complementary memory structures: Artifact Memory, Action Graph, and Feedback Memory.

Figure **??** conceptually illustrates the architecture.

The reasoning engine receives a user request together with context retrieved from the three memory structures. Based on the retrieved information, the reasoning engine determines one or more executable actions. Following execution, observations are recorded as feedback and incorporated back into the adaptive memory. This continuous update process allows future decisions to benefit from accumulated experience.

Unlike architectures that tightly couple memory with a specific reasoning algorithm, the proposed architecture separates adaptive memory from reasoning. Consequently, the reasoning engine may consist of a rule-based system, a Large Language Model, a future learning algorithm, or a human operator. Regardless of the implementation, all reasoning engines interact with the same adaptive memory interface.

The architecture consists of three primary components.

4.1 Artifact Memory

Artifact Memory stores persistent contextual information that may be retrieved during reasoning. Artifacts include both human-generated and machine-generated information such as documents, conversations, source code, structured databases, images, configuration files, and application state.

Unlike conventional storage systems that simply preserve information, Artifact Memory serves as contextual evidence for future reasoning. Retrieval mechanisms may employ semantic search, structured queries, metadata filtering, or other application-specific techniques depending on the underlying implementation.

4.2 Action Graph

The Action Graph represents executable behavior as a directed graph of actions and their relationships. Nodes correspond to executable operations, while edges describe ordering constraints, dependencies, alternative execution paths, or observed transitions.

Rather than representing only predefined workflows, the Action Graph evolves over time as new actions are executed and additional relationships are observed. Historical execution patterns provide valuable context for selecting future actions and coordinating complex tasks across heterogeneous software environments.

4.3 Feedback Memory

Feedback Memory records observations generated during or after execution. Feedback may originate from human evaluations, software systems, environmental sensors, execution metrics, or application-specific performance indicators.

Unlike traditional logging systems, Feedback Memory is intended to influence future reasoning directly. Successful outcomes strengthen confidence in previously executed actions, while failures, corrections, and human preferences become persistent knowledge that guides subsequent decisions.

4.4 Interaction Between Memory Structures

Although each memory structure serves a distinct purpose, they continuously influence one another. Retrieved artifacts provide context for selecting executable actions. Executed actions generate new artifacts and operational history. Feedback evaluates both retrieved information and executed behavior, allowing the adaptive memory to evolve through continual interaction.

The proposed architecture therefore treats artifacts, actions, and feedback as complementary representations of system experience rather than independent software components. This unified representation provides a common adaptive substrate upon which diverse reasoning engines may operate.

5 Adaptive Feedback Loop

The central principle of the Semut Adaptive Architecture is that intelligent behavior emerges through a continuous interaction between reasoning, execution, and adaptive memory. Rather than viewing memory as passive storage, the proposed architecture treats memory as an evolving representation of accumulated system experience.

Each interaction begins with a request from a human user or an external software system. The request itself provides only partial context. Additional information is retrieved from the three complementary memory structures: Artifact Memory, Action Graph, and Feedback Memory. Together, these memories provide the reasoning engine with historical knowledge, executable behavior, and prior observations.

Using the retrieved context, the reasoning engine selects one or more executable actions. The reasoning engine itself is intentionally unspecified by the architecture and may consist of a rule-based system, a Large Language Model, a future learning algorithm, or a human operator. The architecture therefore separates adaptive memory from the reasoning mechanism.

Selected actions are subsequently executed through external software systems, application interfaces, robotic platforms, or human participants. Execution produces observable outcomes, including successful task completion, failures, user corrections, execution metrics, environmental observations, and additional artifacts generated during the task.

These observations are collected as Feedback Memory. Unlike traditional monitoring systems, feedback is not treated solely as diagnostic information. Instead, feedback directly influences future reasoning by updating all three adaptive memory structures.

Successful execution may strengthen confidence in particular action sequences. Failures may introduce alternative execution paths or additional constraints. Newly generated documents become artifacts available for future retrieval. Human corrections become persistent feedback that guides subsequent decisions. Consequently, each execution contributes to the continual evolution of the adaptive memory.

The adaptive loop may therefore be summarized as the following sequence:

1. Receive a request.
2. Retrieve relevant artifacts, actions, and feedback.
3. Reason over the retrieved context.
4. Select executable actions.
5. Execute the selected actions.
6. Observe execution outcomes.
7. Update Artifact Memory, Action Graph, and Feedback Memory.
8. Repeat for subsequent requests.

Unlike static workflows, the proposed architecture continuously incorporates accumulated experience into future decision making. Rather than separating retrieval, execution, and evaluation into independent subsystems, the adaptive loop provides a common mechanism through which intelligent systems progressively improve their contextual understanding and operational behavior over time.

Although the architecture does not prescribe a particular learning algorithm, it establishes a unified interface through which future reasoning engines may utilize accumulated experience. As reasoning capabilities evolve, the adaptive memory remains compatible because its responsibilities are independent of the underlying reasoning implementation.

6 Implementation Considerations

The Semut Adaptive Architecture is intended as a conceptual systems architecture rather than a software framework tied to a particular implementation. Consequently, the proposed memory structures may be implemented using a variety of databases, workflow engines, reasoning systems, and execution environments.

6.1 Artifact Memory

Artifact Memory may be implemented using conventional document databases, relational databases, object storage systems, vector databases, or combinations thereof. Depending on the application domain, artifacts may include documents, source code, emails, conversations, configuration files, images, structured records, or other persistent information.

Retrieval mechanisms are similarly implementation dependent. Systems may employ keyword search, semantic retrieval, metadata filtering, graph traversal, or hybrid retrieval strategies. The proposed architecture requires only that relevant contextual information be retrievable before reasoning begins.

6.2 Action Graph

The Action Graph represents executable behavior independently from the underlying execution environment. Actions may correspond to API calls, software functions, robotic operations, human tasks, workflow nodes, or external services.

Graph edges represent observed relationships between actions, including ordering constraints, dependencies, alternative execution paths, and historical transitions. As additional executions occur, the graph continuously evolves by incorporating newly observed relationships and execution outcomes.

6.3 Feedback Memory

Feedback Memory aggregates observations produced throughout system operation. Feedback may originate from multiple sources, including human approval, software-generated metrics, execution status, application logs, environmental sensors, or explicit user corrections.

Rather than serving exclusively as monitoring data, Feedback Memory becomes part of the adaptive memory available during future reasoning. This enables previously successful behavior, recurring failures, and accumulated user preferences to influence subsequent decision making.

6.4 Reasoning Engine Independence

A defining characteristic of the proposed architecture is the separation between adaptive memory and reasoning. The architecture therefore does not prescribe a particular reasoning algorithm.

Possible reasoning engines include:

- Rule-based systems
- Large Language Models
- Classical planning algorithms
- Reinforcement learning systems

- Future foundation models
- Human decision makers

Each reasoning engine interacts with the same adaptive memory through retrieval and memory update operations, allowing reasoning algorithms to evolve independently from the underlying memory architecture.

6.5 Execution Layer

Execution is likewise independent of the adaptive memory. Actions may be performed through software APIs, workflow platforms, command-line tools, robotic systems, web applications, or direct human interaction.

This separation allows the proposed architecture to coordinate heterogeneous software environments without requiring modifications to existing execution platforms.

Overall, the Semut Adaptive Architecture provides a common adaptive interface connecting persistent knowledge, executable behavior, and accumulated feedback while remaining independent of the specific technologies used to implement each component.

7 Architectural Evaluation

This work introduces an architectural abstraction rather than a new reasoning algorithm. Consequently, the evaluation focuses on architectural properties of the proposed system instead of quantitative benchmarking. Future implementations may evaluate execution time, retrieval quality, memory utilization, or task completion performance using application-specific datasets.

7.1 Architectural Objectives

The proposed architecture was designed to satisfy the following objectives.

- Separation of reasoning from persistent memory.
- Unified representation of artifacts, executable actions, and accumulated feedback.
- Continuous adaptation through iterative memory updates.
- Compatibility with heterogeneous software systems.
- Extensibility to future reasoning engines.

These objectives motivate the design decisions presented throughout the previous sections.

7.2 Modularity

Each memory structure has a clearly defined responsibility. Artifact Memory manages persistent contextual information. Action Graph represents executable behavior. Feedback Memory records observations and evaluations.

Because these components communicate through well-defined interfaces, implementations may replace individual modules without modifying the overall architecture. For example, a vector database may replace a relational database for Artifact Memory, or a different workflow engine may replace the execution layer while preserving the remaining adaptive memory structure.

7.3 Reasoning Independence

The architecture intentionally avoids dependence upon any specific reasoning algorithm. Rule-based systems, Large Language Models, planning algorithms, reinforcement learning systems, or future reasoning engines may all retrieve context from the same adaptive memory.

This separation reduces coupling between memory management and reasoning, allowing improvements in reasoning algorithms without requiring changes to the adaptive memory architecture.

7.4 Continuous Adaptation

Unlike architectures in which execution history remains isolated within operational logs, the proposed adaptive loop continuously incorporates execution outcomes into persistent memory.

Successful executions, observed failures, user corrections, and newly generated artifacts become available for future retrieval. Consequently, system behavior evolves through accumulated operational experience rather than relying exclusively on static workflows or predefined knowledge.

7.5 Scalability Considerations

As interactions increase, both Artifact Memory and Action Graph are expected to grow continuously. Practical implementations may therefore require retrieval policies, summarization techniques, memory pruning, graph compression, or hierarchical storage mechanisms to maintain efficient reasoning.

The proposed architecture intentionally separates these implementation strategies from the architectural abstraction itself, allowing different applications to adopt storage and retrieval techniques appropriate to their operational requirements.

7.6 Limitations

The proposed architecture does not specify a retrieval algorithm, planning algorithm, or learning algorithm. Instead, it provides a common adaptive framework within which these components may operate.

Furthermore, maintaining large adaptive memories introduces challenges related to storage growth, retrieval efficiency, privacy, security, and credit assignment. Addressing these challenges represents an important direction for future research.

Overall, the Semut Adaptive Architecture demonstrates how artifacts, executable actions, and accumulated feedback may be organized within a unified adaptive memory while remaining independent of the underlying reasoning engine.

8 Discussion

The Semut Adaptive Architecture is intended as an architectural abstraction rather than a replacement for existing intelligent systems. Retrieval systems, workflow engines, knowledge graphs, large language models, and agent frameworks each address important aspects of intelligent behavior. The proposed architecture instead provides a common adaptive memory framework through which these components may interact more consistently.

A distinguishing characteristic of the architecture is the explicit separation between adaptive memory and reasoning. Existing intelligent systems frequently couple memory management with a particular reasoning engine or application implementation. In contrast, the proposed architecture treats Artifact Memory, Action Graph, and Feedback Memory as persistent system components that remain available regardless of the reasoning mechanism employed. As reasoning algorithms continue to evolve, accumulated system experience may therefore remain reusable without redesigning the surrounding memory architecture.

Another important implication concerns continual adaptation. Traditional software workflows often execute predefined sequences of operations, while retrieval systems primarily provide contextual information for reasoning. The proposed architecture instead allows every interaction to contribute additional knowledge to the adaptive memory. Newly generated artifacts, successful execution patterns, observed failures, and human corrections all become part of future system behavior. Consequently, adaptation occurs through the continual refinement of the memory structures rather than exclusively through updates to the reasoning engine.

The separation of Artifact Memory, Action Graph, and Feedback Memory also provides opportunities for heterogeneous software integration. Different applications may maintain their own storage technologies, execution environments, and reasoning systems while exchanging information through a common adaptive representation. Such separation may simplify coordination across enterprise software, personal assistants, robotic systems, scientific workflows, and distributed multi-agent environments.

The architecture also aligns naturally with human-in-the-loop systems. Human users are not limited to issuing requests; they continuously contribute feedback, corrections, preferences, and evaluations that become persistent components of adaptive memory. Rather than viewing humans solely as supervisors

of intelligent systems, the proposed architecture considers human interaction an integral source of continual adaptation.

Despite these advantages, several challenges remain. Long-term adaptive memories require efficient retrieval mechanisms, scalable storage strategies, effective credit assignment, and mechanisms for preserving privacy and security. As adaptive memories continue to grow, summarization, hierarchical organization, and selective retrieval may become increasingly important implementation considerations. These challenges are independent of the proposed architectural abstraction but will influence practical deployments.

Overall, the Semut Adaptive Architecture should be viewed as a systems-level organizational framework rather than a competing reasoning algorithm. Its primary objective is to provide a unified adaptive memory structure capable of supporting diverse reasoning engines while enabling continual adaptation through accumulated artifacts, executable actions, and feedback.

9 Future Work

The Semut Adaptive Architecture establishes a conceptual framework for integrating artifacts, executable actions, and accumulated feedback within a unified adaptive memory. Numerous opportunities remain for extending both the architecture and its practical implementations.

One important direction involves retrieval optimization. As adaptive memories continue to grow, efficiently identifying the most relevant artifacts, actions, and feedback will become increasingly important. Future research may investigate hierarchical retrieval strategies, graph-based search, adaptive indexing, memory summarization, and retrieval policies learned from previous system behavior.

Another area of interest concerns automatic feedback interpretation. The current architecture assumes that execution outcomes, human evaluations, and environmental observations are available for incorporation into Feedback Memory. Future systems may employ machine learning techniques to automatically estimate confidence, identify recurring execution patterns, detect anomalies, and determine which observations should influence future decision making.

The evolution of the Action Graph also presents several research opportunities. Rather than recording only observed execution sequences, future implementations may infer alternative execution paths, estimate action reliability, identify reusable workflows, and automatically discover higher-level task abstractions from repeated operational behavior.

Multi-agent coordination represents another promising application. Multiple reasoning engines or intelligent assistants may share portions of Artifact Memory, Action Graph, and Feedback Memory while maintaining independent execution environments. Such distributed adaptive memories may enable collaborative problem solving across heterogeneous software systems.

Future implementations should also investigate practical deployment issues, including privacy preservation, access control, memory ownership, synchroniza-

tion across distributed systems, and long-term storage management. As adaptive memories accumulate increasingly sensitive operational information, appropriate governance mechanisms will become essential.

Finally, comprehensive empirical evaluation remains an important direction for future work. Prototype implementations may compare the proposed architecture against existing intelligent systems across domains such as enterprise automation, software engineering, robotics, scientific workflows, and personal AI assistants. Such evaluations may investigate task completion rates, adaptation efficiency, retrieval quality, memory utilization, and long-term operational performance.

The architecture presented in this paper provides a foundation for these future investigations while remaining independent of any specific reasoning algorithm or implementation technology.

10 Conclusion

Modern intelligent systems increasingly integrate retrieval systems, workflow engines, persistent memory, reasoning models, and human feedback. Although these components have individually advanced, they frequently maintain separate representations of system state, limiting their ability to continually adapt through accumulated operational experience.

This paper introduced the Semut Adaptive Architecture, a unified architectural framework organized around three complementary memory structures: Artifact Memory, Action Graph, and Feedback Memory. Rather than treating these components as isolated subsystems, the proposed architecture integrates them through a continuous adaptive feedback loop in which every execution contributes additional contextual information for future reasoning.

A distinguishing characteristic of the proposed architecture is its independence from any particular reasoning engine. Rule-based systems, Large Language Models, future learning algorithms, and human operators may all utilize the same adaptive memory while remaining free to evolve independently. By separating adaptive memory from reasoning, the architecture seeks to provide a stable foundation capable of supporting heterogeneous intelligent systems over time.

The primary contribution of this work is therefore architectural rather than algorithmic. Instead of introducing a new reasoning model, the proposed framework provides a common abstraction through which artifacts, executable actions, and accumulated feedback may be represented as complementary forms of adaptive memory. This unified representation offers a conceptual foundation for intelligent assistants, enterprise automation, software engineering agents, robotics, and future multi-agent systems.

As intelligent systems continue to expand across increasingly diverse software environments, architectures capable of preserving long-term experience while remaining independent of individual reasoning technologies may become increasingly valuable. The Semut Adaptive Architecture represents one possi-

ble step toward such adaptive systems by organizing memory, execution, and feedback within a coherent and extensible architectural framework.

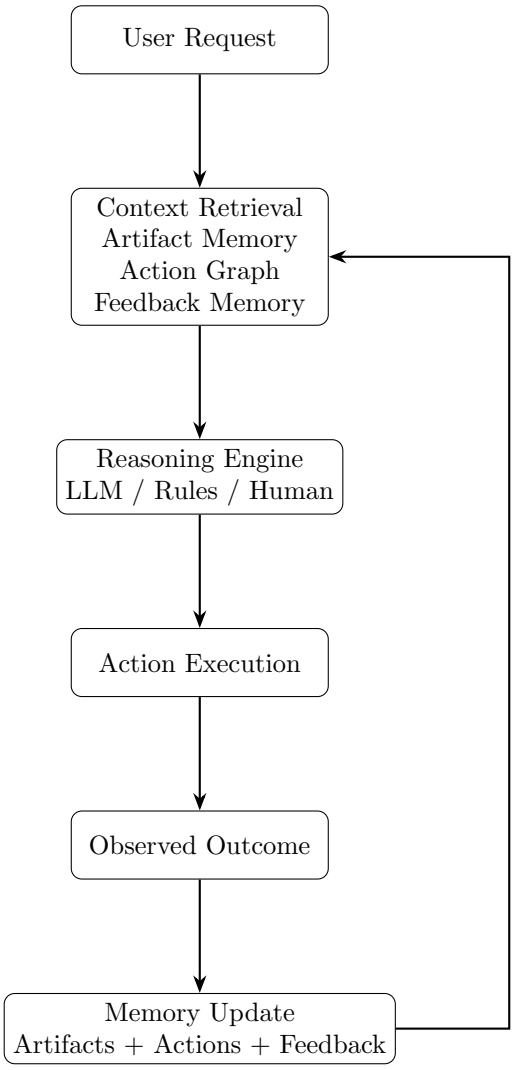

Figure 1: The Semut adaptive feedback loop. Each interaction retrieves context from Artifact Memory, Action Graph, and Feedback Memory, executes selected actions, observes outcomes, and updates all memory structures for future reasoning.